Node.js + Express + MongoDB Backend API

Server.js

Prepared by : Mathuppriya Naguleswaran.

Node.js + Express + MongoDB Backend API

The languages/technologies involved are:

1. JavaScript (Core language)
2. Node.js (Runtime)
3. Express.js (Framework)
4. MongoDB (Database)
5. Mongoose (ODM)

JavaScript : Programming language used to create logic, handle data, control behavior

- Originally Runs in browser.
- with Node.js Runs on server too.
- Same language for frontend + backend
- Easy to learn
- Huge ecosystem

1. Variables : To store data in memory

```
let name = "Mathu";
```

2. Functions : Reuse logic , Organize code

```
function add(a, b) {
return a + b;
}
```

3. Objects : To store structured data (like database records)

```
const student = {
name: "John",
age: 22
};
```

4. Async / Await : Database operations take time JavaScript handles it asynchronously

```
const data = await Student.find();
```

NODE.JS : (SERVER ENGINE)

- A runtime environment that allows JavaScript to run outside browser
- Without Node: JavaScript only works in browser
- With Node:
  - ✓ build backend
  - ✓ handle requests
  - ✓ connect database
- How Node Works : It uses **Event Loop** , **Non-blocking I/O** (Can handle MANY users at once)
- `const http = require('http');` (Node gives built-in modules)

EXPRESS.JS

- A framework built on Node.js
- Used to create APIs easily
- Without Express: Writing server is complex
- With Express:
  ✓ simple
  ✓ fast
  ✓ clean routing

1. Server Creation : Creates server instance

```
const express = require('express');
const app = express();
```

2. Middleware : Converts request body → JSON
(Without this, req.body is undefined)

```
app.use(express.json());
```

3. Routes : Handle client requests

```
app.get('/', (req, res) => {
res.send("Hello");
});
```

MongoDB : NoSQL database

Stores data as documents (JSON-like)

- Flexible
- Fast
- Good for real-time apps
- In SQL DB we called tables but here called as collections. Likewise Rows = Documents.

Mongoose : Tool to connect Node.js with MongoDB

- MongoDB alone: No structure, No validation
- Mongoose:
  ✓ Schema
  ✓ Validation
  ✓ Relationships

```
const studentSchema = new mongoose.Schema({
name: String,
email: String
});
```

Why Schema : Even though MongoDB is flexible

We still need rules: required fields, data types, validation

JavaScript → Language
↓
Node.js → Runs JS on server Kitchen system
↓
Express → Handles requests Waiter
↓
Mongoose → Connects DB Recipe rules
↓
MongoDB → Stores data Storage room

JavaScript as the core programming language, executed on the server using Node.js. Express.js is used as a lightweight web framework to build RESTful APIs. MongoDB is used as a NoSQL database to store data in JSON-like documents, and Mongoose is used as an Object Data Modeling (ODM) library to enforce schema structure and validation. These technologies work together to create a scalable backend system capable of handling CRUD operations and analytics queries.

---

server.js

server.js is the **main entry point**, where you create the Express app, add middleware, define a root route, attach route modules, and connect to MongoDB.

server.js : *server.js is the* ***starting file of the backend****.*

When you run in the terminal

```
node server.js
```

- Node.js starts reading this file from top to bottom.
- So this file is responsible for:
  - ✓ loading packages
  - ✓ creating the server
  - ✓ configuring middleware
  - ✓ connecting the database
  - ✓ attaching routes
  - ✓ starting the app

Think of it like the **control room** of the backend.

Without server.js:

- there is no running server
- no route can be accessed
- no database connection is started
- Postman cannot talk to your project

So even if you create models and routes, they are useless until server.js brings everything together.

```
JS server.js > ...
1   require('dotenv').config();
2
3   const express = require('express');
4   const mongoose = require('mongoose');
5   const cors = require('cors');
6
7   const studentRoutes = require('./routes/students');
8   const menuItemRoutes = require('./routes/menuItems');
9   const orderRoutes = require('./routes/orders');
10  const analyticsRoutes = require('./routes/analytics');
11
12  const app = express();
13
14  const PORT = process.env.PORT || 3000;
15  const MONGO_URI = process.env.MONGO_URI;
16
```

```
17  // middleware
18  app.use(cors());
19  app.use(express.json());
20
21  // root route
22  app.get('/', (req, res) => {
23    res.json({ message: 'Campus Food API is running' });
24  });
25
26  // attach routes
27  app.use('/students', studentRoutes);
28  app.use('/menu-items', menuItemRoutes);
29  app.use('/orders', orderRoutes);
30  app.use('/analytics', analyticsRoutes);
31
32  // connect DB and start server
33  mongoose
34    .connect(MONGO_URI)
35    .then(() => {
36      console.log('✅ Connected to MongoDB');
37      app.listen(PORT, () => {
38        console.log(`🚀 Server running on http://localhost:${PORT}`);
39      });
40    })
41    .catch((err) => {
42      console.error('❌ MongoDB connection error:', err.message);
43    });
```

1. Load environment variables

```
require('dotenv').config();
```

- This loads values from a .env file into process.env
- Because sensitive or changeable values should not be hardcoded.(database URL , port number , secret keys)
  - ✓ If you use hardcording for the changeable data ,its become less flexible, because you wants to edit code.
  - ✓ So, Using environment variables makes the app configurable.
- For this line code work, you must install dotenv => **npm install dotenv**

2. Import Express

```
const express = require('express');
```

- This imports the Express library.
- Express is a Node.js framework used to build web servers and APIs.
- Node.js alone can create a server, but Express makes it much easier.
- Without Express more code, harder routing, more manual setup .
- With Express easier routes, middleware support, cleaner structure.
- So Express helps us build APIs faster and more cleanly.

3. Import Mongoose

```
const mongoose = require('mongoose');
```

- Mongoose is the tool that helps Node.js communicate with MongoDB.
- MongoDB stores data, but Mongoose gives the schemas, validation, models, easier queries.
- So Mongoose acts like a bridge between JavaScript code and MongoDB database.

4. Import CORS

```
const cors = require('cors');
```

- This imports the CORS middleware.
- CORS stands for **Cross-Origin Resource Sharing**.
- **Why we need it :** Usually frontend and backend may run on different ports, for example:
  - frontend: http://localhost:5173
  - backend: http://localhost:3000
- Even though both are on localhost, the browser treats them as different origins.
- Without CORS: browser blocks frontend requests to backend
- With CORS: backend allows requests from frontend
- Postman usually works even without CORS, because CORS is mainly a browser security rule.
- But in real projects, especially MERN projects, CORS is very important.

```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm install express mongoose cors dotenv

added 85 packages, and audited 86 packages in 14s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\vallu\Desktop\campus-food-api_It24102099>
```

To install all of these we can used this & you can use it with import those libraries in server.js

5. Import route modules

```
const studentRoutes = require('./routes/students');
const menuItemRoutes = require('./routes/menuItems');
const orderRoutes = require('./routes/orders');
const analyticsRoutes = require('./routes/analytics');
```

- These lines import the route files from the routes folder.
- Why we do this : Because each feature has its own endpoints.
- Examples: students routes handle student requests, analytics routes handle reporting endpoints
- Why separate them into files : Because if you write all routes inside server.js, it becomes too long, messy, hard to maintain .
  - ✓ This is called **modular coding**.
  - ✓ A senior engineer always separates code by responsibility.

```
'./routes/students'
```

- . means current folder
- ./routes means go into routes folder
- students means file students.js

6. Create the Express app

```
const app = express();
```

- This creates the Express application.
- Why it matters : app is now your main server object.
- **You use app to: add middleware, define routes, start the server**

7. Read configuration values

```
const PORT = process.env.PORT || 3000;
const MONGO_URI = process.env.MONGO_URI;
```

Reads values from environment variables.

const PORT = process.env.PORT || 3000;

- use process.env.PORT if it exists
- otherwise use 3000
- **|| 3000 :** Because if .env is missing or PORT is not defined, the app still works on port 3000. This is called a **default fallback**.

const MONGO_URI = process.env.MONGO_URI;

- This gets the database connection string.
- Example: MONGO_URI=mongodb://127.0.0.1:27017/campus_food
- Why this : Without a valid MongoDB URI: database cannot connect, app cannot store or retrieve data.

Middleware section

Middleware is one of the most important backend concepts. A middleware is a function that runs during the request-response cycle. It can:

- modify request
- modify response
- validate data
- parse data
- allow or block requests
- pass control to next step

8. Enable CORS middleware

```
app.use(cors());
```

- Adds CORS middleware to the app.
- app.use(): tells Express to use a middleware for incoming requests.
- Why this middleware matters : When a frontend sends request to backend from another origin, this middleware helps allow it.

9. Enable JSON body parsing

```
app.use(express.json());
```

- This middleware teaches the server how to understand JSON.
- This middleware reads JSON data from the request body and converts it into a JavaScript object.

Example

If Postman sends:

```JSON
{
  "name": "Alice",
  "email": "alice@example.com"
}
```

then after `express.json()`:

```JavaScript
req.body
```

becomes:

```JavaScript
{ name: 'Alice', email: 'alice@example.com' }
```

- Without this middleware:
  - req.body will often be undefined
  - POST and PATCH requests become hard to use

10. Create root route

```
app.get('/', (req, res) => {
res.json({ message: 'Campus Food Ordering API is running' });
});
```

app.get() : handle HTTP GET request for the path /

(req, res) : req = request object , res = response object

Why this route is useful : It helps test whether the server is alive.

If this works, your server is running correctly.

Why respond with JSON : Because this is an API project, so JSON response is standard.

11. Attach routes

```
app.use('/students', studentRoutes);
```

1. Mount student routes

- Any request starting with /students should be handled by studentRoutes**.**
- For Example **:**
    1. If students.js contains: **router.post('/');**
        - then the full endpoint becomes: **POST /students**
    2. If it contains: **router.get('/:id');**
        - then the full endpoint becomes: **GET /students/:id**
- **You define only relative paths inside the route file, and server.js gives the base path.**

2. Mount menu item routes

```
app.use('/menu-items', menuItemRoutes);
```

- This makes route file endpoints available under /menu-items.
- Example: router.get('/'); becomes: GET /menu-items

3. Mount order routes

```
app.use('/orders', orderRoutes);
```

- This connects all order-related routes.
- Examples: POST /orders
    - GET /orders
    - GET /orders/:id

4. Mount analytics routes

```
app.use('/analytics', analyticsRoutes);
```

This connects analytics endpoints like:

- /analytics/total-spent/:studentId
- /analytics/top-menu-items
- /analytics/daily-orders

Database connection

mongoose
.connect(MONGO_URI)

12. Start MongoDB connection

- The code tries to connect to MongoDB using the connection string.
- .connect() is asynchronous : bcoz connecting to a database takes time. So Mongoose returns a Promise.

  That is why we use:
  - .then()
  - .catch()
- Why connect before starting server? : Because if database is not connected, many routes will fail.
- **A clean backend should ideally start only after DB is ready.**

If connection succeeds

```
.then(() => {
console.log('MongoDB connected successfully');

app.listen(PORT, () => {
console.log(`Server is running on port ${PORT}`);
});
})
```

.then()	It runs if the database connection succeeds.
log success message	This helps developers know: • DB is connected • setup is correct
app.listen()	Because we want to start the server only after MongoDB connects successfully.
PORT	The port is the communication channel where your server listens. If port is 3000, the app is reachable at: http://localhost:3000
${PORT}`)	

If connection fails

```
.catch((error) => {
console.error('MongoDB connection failed:',
error.message);
});
```

.catch()	Handles connection errors. Because many things can go wrong: • wrong URI • MongoDB server not running • no internet for Atlas • authentication failure
error.message	it prints the useful human-readable part of the error.

Full backend life cycle.

Step 1	Node starts reading server.js
Step 2	Packages are imported
Step 3	Express app is created
Step 4	Middleware is attached
Step 5	Routes are mounted
Step 6	MongoDB connection starts
Step 7	If DB connects successfully, server starts listening
Step 8	User sends request from Postman
Step 9	Express receives request
Step 10	Matching route file handles request
Step 11	Route uses model
Step 12	Mongoose talks to MongoDB
Step 13	Response returns to Postman

In server.js is written in this order : Order matters.

A good structure is:

1. load env
2. import packages
3. import routes
4. create app
5. config values
6. middleware
7. test route
8. attach routes
9. connect DB
10. start server

Why middleware before routes? : Because routes may need parsed JSON and CORS support.

Why routes before listen? : Because when server starts, all route configuration should already be ready.

Why DB connect before app.listen? : Because many APIs depend on DB.

This order is not random. It is intentional architecture.

In server.js, I initialize the Express application and configure middleware such as CORS and JSON body parsing to handle incoming requests properly. Then I mount the modular route handlers for students, menu items, orders, and analytics, which keeps the backend architecture clean and maintainable. Finally, I connect to MongoDB through Mongoose and start the server only after a successful connection, ensuring reliable database-driven API execution.